Image classification in industry 4.0

ALESSANDRO MASSIMO MERMALI (136584)

Deep Learning Course 2025-2026

Contents

1 Introduction **4**
1.1 Problem Statement 4
1.2 Objectives 4

2 Background **5**
2.1 Casting Process and Common Irregularities 5
2.2 Economic Impact and Manual Limitations 5
2.3 Deep Learning Solution 5

3 Data **6**
3.1 Data Analysis 6
3.1.1 Dataset composition 6
3.1.2 Data Distribution and Splitting 6
3.1.3 Visual Characteristics 6
3.2 Preprocessing and Normalization 7
3.3 Data Augmentation 8

4 Methodology **9**
4.1 Pipeline Components 9
4.2 Training and Evaluation 10
4.3 Command-Line Interface (CLI) 10
4.3.1 Available Arguments 10
4.3.2 Modes 11
4.3.3 Examples 11

5 Experiments and Results **13**
5.1 Architectural Configurations 13
5.2 Training Dynamics 16
5.3 Final Evaluation and Model Selection 16

6 Conclusion **18**

Abstract

In the context of Industry 4.0, automating quality assurance is critical for reducing manufacturing costs and improving production efficiency. This project addresses the challenge of industrial defect detection by implementing a binary classification of defective and non-defective casting products. Utilizing a real-life industrial dataset of casting images, I developed and evaluated a deep learning model to automate the visual inspection process. My approach leverages convolutional neural networks (CNNs) to extract complex spatial feature from top-view images of metal components. Various architectures were explored, ranging from simple mlp-based architectures to more complex CNN-based. The results demonstrate the efficacy of deep learning in achieving high accuracy and robustness in quality control. All experiments were conducted following open science principles to ensure code reproducibility and transparency.

1 Introduction

The era of **Industry 4.0** has established a new paradigm for manufacturing that integrates Artificial Intelligence (AI) and Internet of Things (IoT) to create "smart factories". A core component of this evolution is Automated Optical Inspection (AOI). Traditional quality control often relies on manual inspection, which is not only labor-intensive but also prone to human error and fatigue.

This project addresses the challenge of industrial defect detection by implementing a binary image classification system. My primary goal is to develop a deep learning based pipeline capable of distinguishing between defective and non-defective casting products with high precision.

1.1 Problem Statement

In metal casting, defects as blowholes, cracks and inclusions can compromise the structural integrity of a component. Detecting these anomalies at the end of production line is crucial to prevent faulty parts from reaching the assembly stage. The challenge lies in developing a robust binary classification system that can operate with high accuracy and low latency in a real-time industrial environment.

1.2 Objectives

The primary goal of this work is to design and implement a neural network pipeline. Specifically the project aims to analyze and preprocess a real-world dataset of casting images (Section 3), define a common methodology for training and evaluation (Section 4) and evaluate deep learning architectures to find the optimal balance between efficiency and efficacy (Section 5).

2 Background

2.1 Casting Process and Common Irregularities

Casting is a fundamental manufacturing process in which a liquid material is poured into a mold containing a hollow cavity of the desired shape and allowed to solidify. In metal industries, the process requires the melting of the metal and the solidification through cooling down.
Despite its versatility, the process is susceptible to various casting defects. The most common ones include:

- **Gas porosity:** blowholes and pinholes caused by trapped gas.
- **Shrinkage defects:** caused by the metal contracting during solidification.
- **Metallurgical defects:** resulting from improper cooling or impurities.
- **Molding material defects:** such as cuts, washes or fusion issues.

2.2 Economic Impact and Manual Limitations

In the modern manufacturing landscape, casting defects represent a significant economic burden. Currently, many realities rely on manual vision inspection. Despite this, this traditional approach faces several critical limitations:

- **Subjectivity:** human inspectors may have inconsistent standards.
- **Fatigue:** inspection is a repetitive task that induces fatigue into the human operator. This may lead to an accuracy drop.
- **Financial loss:** a single defective component in a batch may lead to the rejection of an entire order, causing massive losses.

2.3 Deep Learning Solution

To overcome these challenges, I propose a transition from manual to **Automated Optical Inspection** (AOI). By utilizing Deep Learning techniques and Convolutional Neural Networks (CNNs) the inspection process becomes objective, scalable and more accurate. This project explores how these models can be trained on top-view industrial images to automate quality assurance in a real-world setting.

3 Data

3.1 Data Analysis

The dataset utilized in this project was provided by R. Dabhi [1] on Kaggle. It consists of industrial images of submersible pump impellers. To ensure high-quality data suitable for deep learning, a specialized physical arrangement was constructed for image acquisition. This setup featured controlled lighting to minimize shadows and reflections that could be misidentified as defects. All images were captured from a consistent top-view perspective to maintain spatial uniformity across the classes.

3.1.1 Dataset composition

The datasets is presented in two distinct versions.

- **Original dataset:** high-resolution (512x512 pixels) grayscale set containing 1300 unique images.
- **Augmented dataset:** a collection of lower-resolution (300x300 pixels) grayscale images. These images have been pre-processed with various augmentation techniques, like rotations, flips and shifts, obtaining a dataset of 7348 images.

3.1.2 Data Distribution and Splitting

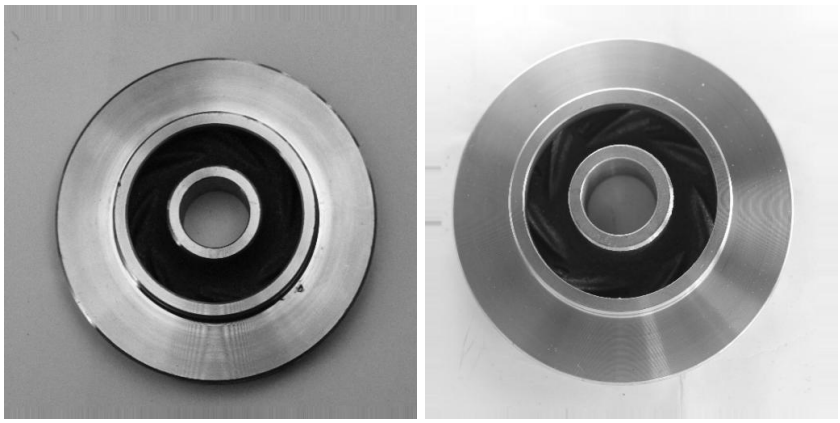
Both the original and augmented dataset feature two different folders used to differentiate between non-defective (`ok_front`) and defective images (`def_front`). The augmented dataset is further split into training and testing folders. The distribution of the dataset is as follows:

Category	`ok_front`	`def_front`	**Total**
512x512	519	781	**1300**
300x300 training set	2875	3758	**6633**
300x300 testing set	262	453	**715**
Total	**3656**	**4992**	**8648**

3.1.3 Visual Characteristics

The images are grayscale, which reduces complexity while retaining the essential textural information required to correctly identify surface irregularities.

In Figure 1 are presented two examples of defective and correct samples within the dataset. As it's visually obvious, the correct sample represents a flawless impeller surface under controlled lighting, whereas the defective ones contain blowholes, pinholes, burrs or shrinkage anomalies.



(a) Defective sample. (`def_front`) (b) Correct sample. (`ok_front`)

Figure 1: Visual comparison between defective and correct samples.

3.2 Preprocessing and Normalization

Data preprocessing is a critical step in the deep learning pipeline, ensuring that the raw image data is transformed into a standardized mathematical format suitable for neural networks. To guarantee optimal convergence and reliable model evaluation, a strict preprocessing pipeline was applied to the dataset.

Resizing The first preprocessing step involved uniformly resizing all images to a fixed spatial resolution of 300×300 pixels. This is done in order to adapt to the characteristics of the processed dataset. Moreover, the size reduction is structurally mandatory, as the neural network architecture requires input tensors of a strictly predefined static dimension. 300×300 was found as the perfect middle ground for both precise detection and computational viability.

Pixel Normalization Following the resizing, pixel intensity normalization was applied. In the raw format, image pixels are represented as 8-bit integers ranging from 0 to 255. Feeding these large unscaled values directly into a neural network may pose significant mathematical challenges: it can lead to instability, erratic gradient updates and immediate saturation of activation layers. To prevent this and ensure a smooth convergence of the loss function, the pixel values were linearly min-max scaled to a precise $[0, 1]$ interval by dividing every pixel by 255.0.

Channel Reduction As mentioned before, the processed dataset is composed of grayscale images. This characteristics is ensured by reducing the number of channels to 1. For this problem, it's not required to use multiple channels as defects appears as abnormal peaks of white or black.

Pipeline Reusability A fundamental principle strictly adhered to during this phase was the absolute consistency of the preprocessing operations. The exact same processing pipeline applied to the training set was applied to both validation and testing sets. Ensuring that the testing set receives the identical scaling as the training set is imperative to prevent artificial data distribution shifts, so guaranteeing the model's performance in a real-world environment. Moreover, the same pipeline used in the training phase was also used for the inference phase, guaranteeing the same treatment for the input data during both phases.

3.3 Data Augmentation

The project initially planned the addition of an augmentation phase as a fundamental part of the preprocessing pipeline. It was later converted into an optional step of the pipeline. **Results presented in the following sections are obtained without augmentation.**
The augmentation is done by randomically applying zoom, rotation, flipping and translation to the images. In Figure 2 are shown variants resulting from the application of data augmentation to a single image.

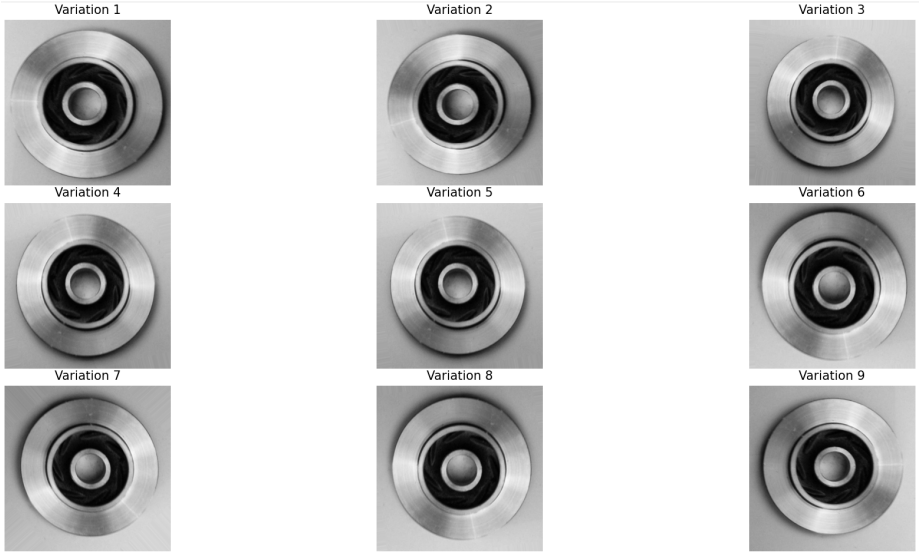


Figure 2: Example of the augmentation task.

4 Methodology

The methodology adopted for this project focuses on modularity and reproducibility. Rather than relying on a single monolithic script, the project was developed as a structured pipeline composed of different components. This approach allows for isolated testing, swapping of model architectures and automated execution of diverse experimental configurations.

4.1 Pipeline Components

The pipeline is divided into distinct and specialized modules, each responsible for a specific phase of the workflow:

- **data**
  - **Data Loader (`loader.py`):** this component loads the dataset into the system, by also extracting their label from the folder name and associating it with the data.
  - **Data Processor (`processor.py`):** this component is responsible for the preprocessing and data augmentation pipeline (Section 3).
- **model**
  - **Model Builder (`builder.py`):** it defines the structure of the used models.
  - **Model Engine (`engine.py`):** it contains most of the logic behind the system. It handles the training, evaluation and inference phase. For the training phase, it compiles the initialized models and integrates essential Keras callbacks, such as `EarlyStopping` or `ModelCheckpoint`.
- **utils**
  - **Evaluator (`evaluator.py`):** a module designed for rigorous model evaluation. This component dynamically computes the optimal decision boundary using the ROC curve and maximizing J statistic ($J = TPR - FPR$). It also computes the evaluation of the models and plots the confusion matrix.
  - **Plotter (`plotter.py`):** an utility module responsible for generating visual artifacts, such as training history curves and comparison. These visualizations are automatically saved to the filesystem at the end of each run.
- **Experiment Handler (`experiment_handler.py`):** this component combines all the components mentioned before in order to build the pipeline desired.

- **Main (`main.py`):** this is the main entry point of the system. In this component it's possible to define the characteristics of the experiments. It also provides a *Command Line Interface* to interact with the system.

4.2 Training and Evaluation

To systematically identify the most suitable algorithm for the binry classification task, a strict experimental protocol was established. Every architecture generated by the Model Builder is subject to an identical training and evaluation protocol: they are trained using the exact same normalized dataset splits and optimized using the **Binary Crossentropy loss** function, which is the standard for two-class estimation.
For the training of the networks, **Stochastic Gradient Descent** paired with **Momentum** was selected as the optimizer. While adaptive algorithms like Adam are known for their faster convergence, they often are more prone to overfitting, as occurred in the initial experimentation phases of this project.
Once the training phase concludes, the Evaluator calculates the optimal classification threshold based on the validation probabilities. The final performance of the model is then measured on the isolated testing set using standardized metrics. This pipeline ensures that any difference in performance is exclusively related to the architectural characteristics of the models, guaranteeing a sound and fair comparison.

4.3 Command-Line Interface (CLI)

To ensure high usability and facilitate automated experimentation, the entire pipeline uses a Command-Line Interface (CLI) implemented via the `argparse` module. The CLI acts as an unified entry point for all operations, including model training, evaluation and inference.

4.3.1 Available Arguments

The CLI accepts various arguments that allow the user to dynamically modify hyper-parameters and define the execution behavior without altering the underlying source code. The arguments (and their default values) are listed on Table **??**.

Argument	Target Mode	Default	Description
`--mode`	–	`train`	Defines the operational phase (`train`, `predict`, `evaluate`, `original_evaluation`).
`--batch_size`	`train`	16	Number of samples propagated through the network before updating internal parameters.
`--lr`	`train`	0.01	Sets the initial learning rate for the optimizer.
`--augment`	`train`	*False*	Boolean flag to activate the data augmentation pipeline on the training set.
`--mp`	`train`	*False*	Boolean flag to activate Mixed Precision (`float16`) training to accelerate computation.
`--model_name`	`predict`, `evaluate`	*None*	Required. Specifies the target model (e.g., `ModelA`) to load from the `models/` directory.
`--image_path`	`predict`	*None*	Specifies the path to a single image for isolated inference.
`--folder_path`	`predict`	*None*	Specifies the path to a directory containing multiple images for batch inference.

Table 1: CLI Arguments

4.3.2 Modes

The CLI allows for multiple modes through the `--mode` argument. Specifically, the modes are:

- **train:** start the training and evaluation phase for all the models in the **experiment** dictionary.
- **evaluate:** start only the evaluation phase for the specified model. Available only if the model specified was trained beforehand.
- **predict:** uses a specified model to predict the class and score of a specified image.
- **original_evaluation:** evaluates a specified model on `data/original`, which contains the original data in the 512x512. Despite not being fully reliable, it gives a general idea on how the model works on not processed data that was never seen before.

4.3.3 Examples

Model training

```
py main.py --mode train --epochs EPOCHS --lr LR --augment --mp
```

Model evaluation

```
py main.py --mode evaluate --model_name MODEL_NAME
```

Inference

```
py main.py --mode predict --model_name MODEL_NAME --image_path IMAGE_PATH
```

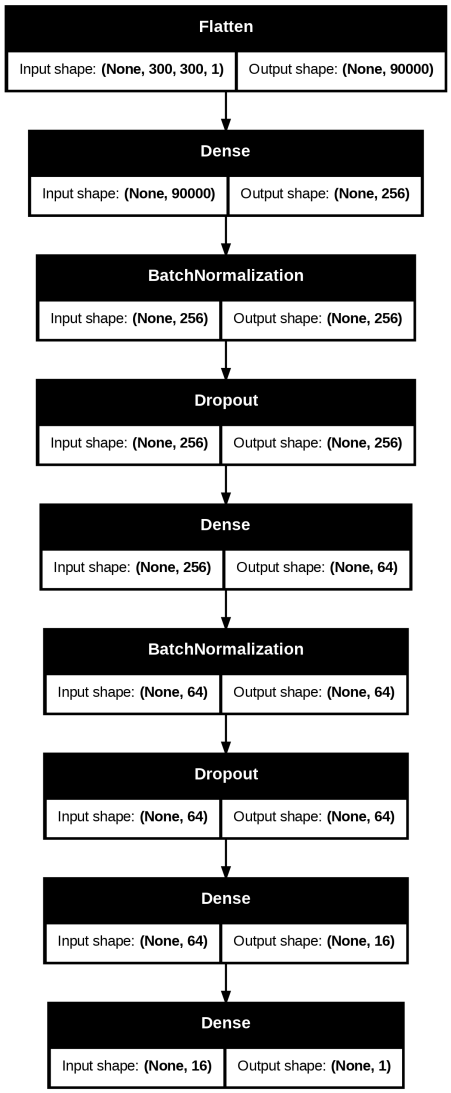
5 Experiments and Results

To identify the optimal deep learning architecture for the automated detection of casting defects, a systematic experimental protocol was executed. Four distinct models were instantiated using the `ModelBuilder` component, trained under identical conditions and evaluated.

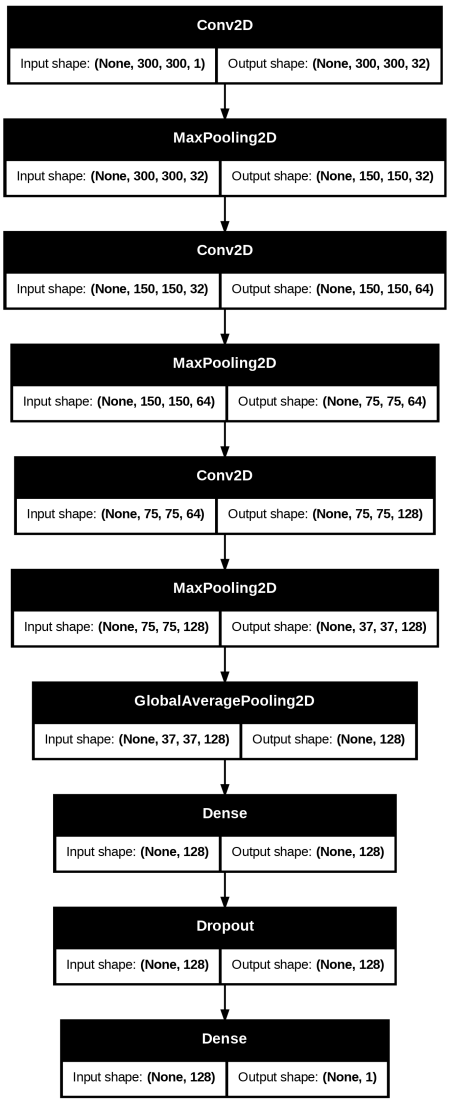
5.1 Architectural Configurations

The tested architectures share standardized hyper-parameters (e.g., Dropout rate of 0.4, L_2 regularization) but differ fundamentally in their structural design and dimensionality reduction strategies:

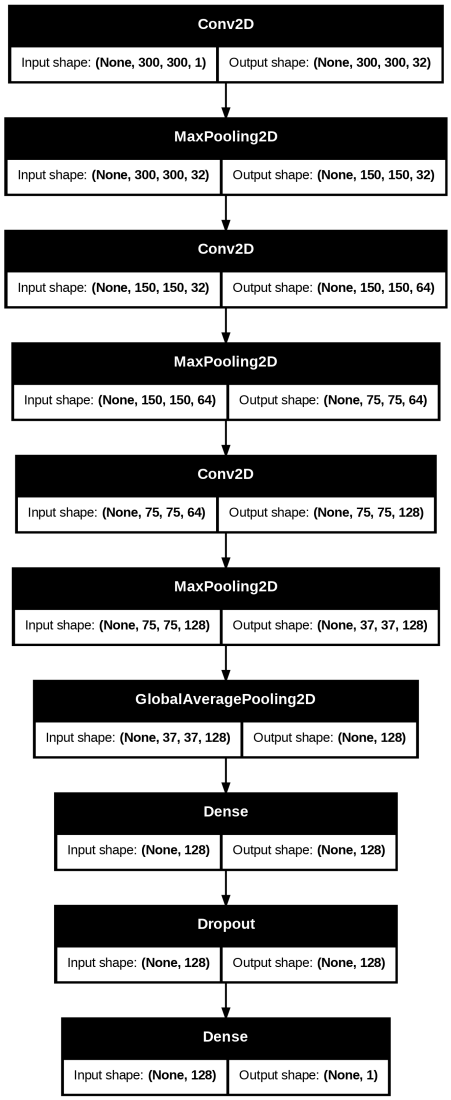
- **Model A (MLP Baseline):** A standard Multi-Layer Perceptron designed to test if raw pixel intensities hold enough linearly separable information without spatial feature extraction. Figure 3a.
- **Model B (CNN + GAP2D):** A Convolutional Neural Network built with three sequential convolutional blocks. The spatial feature maps are reduced using a `GlobalAveragePooling2D` (GAP2D) layer before the classification head, ensuring that the spatial average is extracted when reducing the input to a vector. Figure 3b.
- **Model C (CNN + GMP2D):** Structurally identical to Model B, but utilizes `GlobalMaxPooling2D` (GMP2D) to reduce dimensionality. It aims to extract the single most prominent feature from each feature map rather than the spatial average. Figure 3c.
- **Model D (CNN + Flatten):** Shares the convolutional backbone but uses a standard `Flatten` layer, resulting in a massively unrolled 1D vector before the Dense layers. Figure 3d.



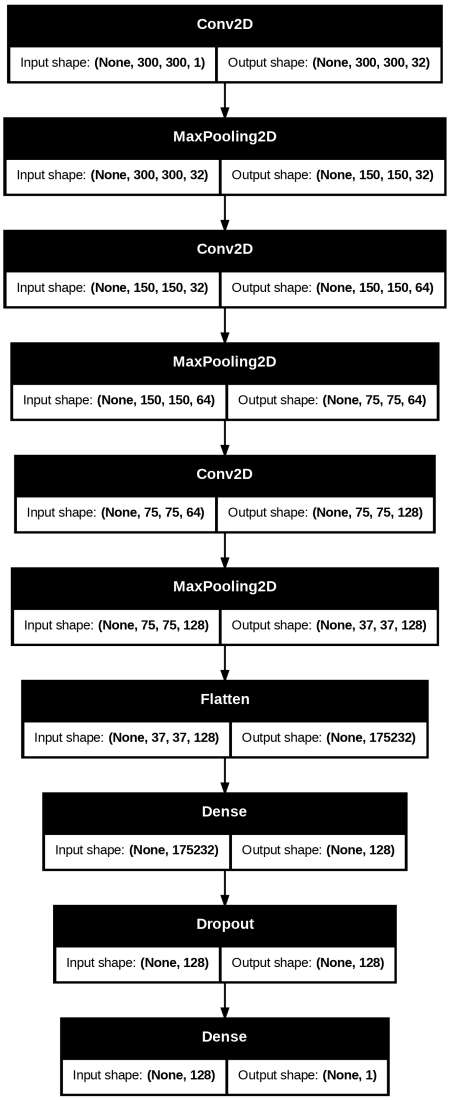
(a) Model A.



(b) Model B.



(c) Model C.



(d) Model D.

Figure 3: Four models.

Time for Training

Experiment	Total Time
ModelA	3m 31s
ModelB	21m 8s
ModelC	20m 56s
ModelD	13m 51s

Figure 4: History comparison between the models.

5.2 Training Dynamics

The training phase revealed significant behavioral differences across the architectures, these differences are plotted in the Figure 4. **Model D** (Flatten) proved highly prone to overfitting due to the explosion of trainable parameters, triggering the Early Stopping at Epoch 10. **Model A** (MLP) performed adequately, acting as a solid baseline, but lacked spatial invariance, failing to generalize the defective features, as seen in Figure 5a.

The most interesting comparison emerged between **Model B** (GAP2D) and **Model C** (GMP2D). During the initial phases of experimentation, **Model B** exhibited highly stable convergence, after the simplification of the model and the application of regularization, it appeared as the most prone to fail in converging, suggesting it relied entirely on overfitting on the previous phases.
On the other hand, **Model C**, while still showing an unstable trajectory, managed to actually converge and didn't show any sign of overfitting in the Evaluation phase. This behavior is theoretically sound: since GMP2D relies entirely on the maximum activation value of a feature map, the gradient updates depend on single peak pixels, making the loss landscape rougher compared to the smoothing effect of GAP2D.

5.3 Final Evaluation and Model Selection

Model C (CNN + GMP2D) is unequivocally selected as the superior architecture.

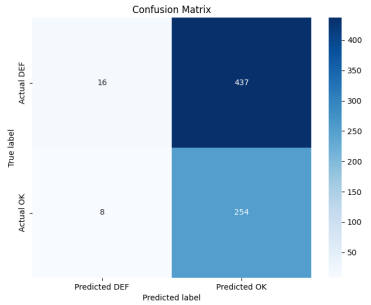
When evaluated with the dynamically optimized threshold on the Test Set (715 instances), **Model C** achieved near-perfect generalization. As demonstrated by its Confusion Matrix (Figure 5b), the model correctly classified 452 out of 453 defective images and 262 out of 262 sound images. This translates to an overall accuracy of **99.86%** and, more importantly, a False Negative rate of strictly zero.

The success of **Model C** lies in the specific inductive bias of the `GlobalMaxPooling2D` layer, which aligns perfectly with the nature of the indus-

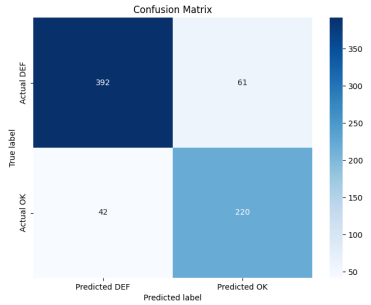
trial task. Casting anomalies such as cracks, pores, or localized shrinkages are typically small, sharp variations concentrated in a tiny fraction of the image.

- If `GAP2D` is used the network averages the strong activation of the localized defect with the near-zero activations of the surrounding healthy metal, effectively diluting the anomaly's signal.
- By using `GMP2D`, the network extracts exclusively the highest activation value. If a specific filter detects a sharp crack, that peak signal is directly propagated to the final `Dense` layer uncompromised.

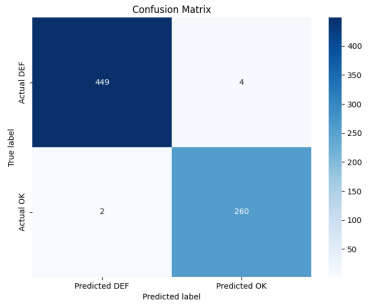
Therefore, despite presenting a more turbulent optimization path during training, the peak-feature extraction of **Model C** proved to be the mathematically correct approach for isolating localized casting anomalies, resulting in a system highly suitable for real-time quality assurance.



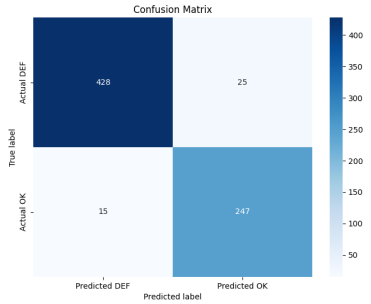
(a) Model A.



(c) Model B.



(b) Model C.



(d) Model D.

Figure 5: Confusion matrices.

6 Conclusion

The study successfully demonstrated the viability and high efficiency of Convolutional Neural Networks for automated visual inspection in an Industry 4.0 context. By developing a highly modular pipeline, I was able to systematically evaluate different architectural hypothese for the binary classification of casting defects.

The experimental phase revealed crucial insights into the behavior of dimensionality reduction layers within CNNs. While Model B (utilizing Global Average Pooling) exhibited the most stable optimization trajectory during training, the rigorous evaluation on the unseen Testing set proved that Model C, utilizing `GlobalMaxPooling2D` (GMP2D), was the superior architecture.

The success of Model C is rooted in its mathematical alignment with the physical nature of casting anomalies. Because defects such as micro-cracks or pores occupy a minimal spatial area with high contrast, the GMP2D layer effectively isolates these peak activations without diluting the signal, a problem observed when averaging the feature maps. Coupled with the dynamic optimization of the classification threshold based on Youden's J statistic, Model C achieved an exceptional overall accuracy of **99.86%**.

Most importantly from an industrial standpoint, the final model achieved a **False Negative rate of strictly zero**. In manufacturing quality control, misclassifying a defective component as sound (False Negative) carries severe financial and reputational costs, far outweighing the cost of a False Positive. The complete elimination of False Negatives makes this architecture not just academically interesting, but directly viable for real-world production lines.

References

[1] Ravirajsinh Dabhi. *casting product image data for quality inspection*. https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product. 2020.

[2] TensorFlow Authors. *Data augmentation*. https://www.tensorflow.org/tutorials/images/data_augmentation. Accessed: 2023-10-24. 2023.